Introduction

Editorial review is one of the longest-lived bottlenecks in scientific writing — and an obvious target for the kind of *reproducible computational workflow* advocated by (Peng 2011). A working draft accumulates structural drift (skipped heading levels, sections that have grown to 2,000 words while their siblings sit at 200), readability drift (a once-clean introduction now reads at the Gunning Fog level of a legal contract (Gunning 1952)), and citation drift (`[@key]` references that no longer exist in `references.bib`, or bibliography entries that nobody actually cites). The traditional remedy — a senior co-author re-reading the manuscript with (Strunk and White 2000) in hand — is slow, non-reproducible, and impossible to apply continuously.

`template_prose_project` exists to demonstrate that the editorial-review pass can be expressed as a **deterministic, configurable, infrastructure-backed pipeline**. This project carries no novel research contribution of its own; its purpose is to show how to compose existing template infrastructure into a complete, reproducible editorial workflow.

The architecture is simple. `manuscript/config.yaml` defines policy: target grade-level band, citation-density floor, heading-structure rules, bibliography-consistency policy. The orchestrator script `scripts/run_prose_pipeline.py` reads the manuscript directory and calls `infrastructure.prose.analyze_manuscript` to produce a `ManuscriptReport`; `src/pipeline/__init__.py::run_prose_pipeline` then cross-checks the cited keys against the `references.bib` (via the

project-owned `src.pipeline.prose_facade.parse_bib_keys` helper), evaluates each configured check, and writes the JSON artefacts, while `src/manuscript/report.py::write_review_report` assembles the markdown review and three diagnostic figures are rendered from the report. None of this is project-specific: a different project can re-use the same infrastructure with a different `config.yaml` and a different manuscript directory.

The remainder of this paper documents the methodology (sec. `sec:methodology`), the run-time results on the bundled manuscript (sec. `sec:results`), and the architectural lessons drawn from wiring prose analysis through the template pipeline (sec. `sec:conclusion`).